

LAB 10 (PL/SQL)

Q1.

(A) Write a PL/SQL block:

- Declare 2 variables
- Add them
- Print result using DBMS_OUTPUT.PUT_LINE

```
set serveroutput on;

-- Q1 (a)
declare
    v_num1 number := 15;
    v_num2 number := 20;
    v_result number;
begin
    v_result := v_num1 + v_num2;
    dbms_output.put_line('Result: ' || v_result);
end;
/
```

Task completed in 0.052 seconds

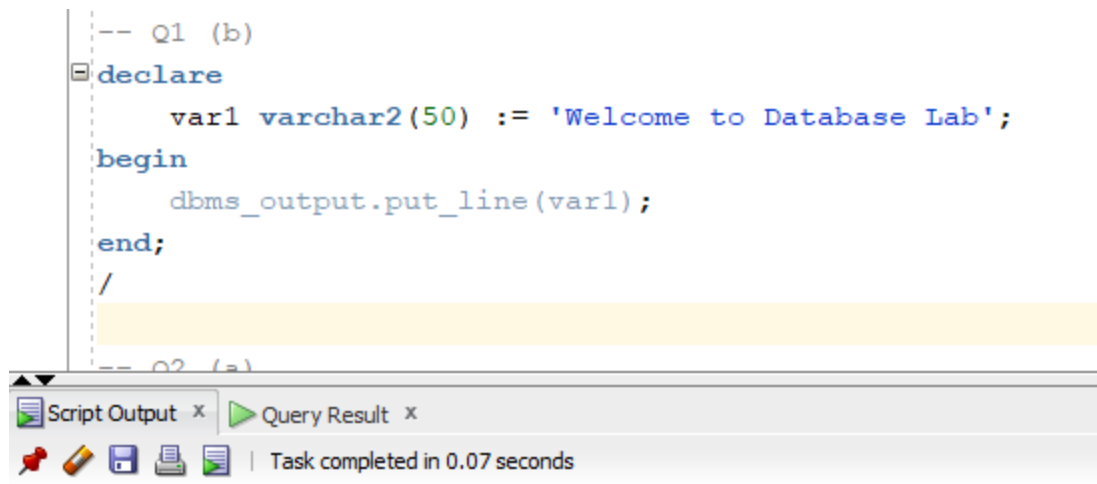
Result: 35

PL/SQL procedure successfully completed.

(B) Write a PL/SQL block:

- Declare a string variable
- Print: "Welcome to Database Lab"

```
-- Q1 (b)
declare
    var1 varchar2(50) := 'Welcome to Database Lab';
begin
    dbms_output.put_line(var1);
end;
/
```

The screenshot shows a SQL IDE window. The top pane contains a PL/SQL script for Q1 (b). The script declares a variable 'var1' of type 'varchar2(50)' with the value 'Welcome to Database Lab'. It then uses 'dbms_output.put_line' to print the value. The bottom pane shows the 'Script Output' tab with the message 'Result: 35'. The 'Query Result' tab is also visible but empty. A status bar at the bottom indicates 'Task completed in 0.07 seconds'.

Result: 35

PL/SQL procedure successfully completed.

Welcome to Database Lab

PL/SQL procedure successfully completed.

Q2.

(a) A company wants to check employee salary:

- If salary > 50,000 → print "High Salary"
- Else → print "Normal Salary"

```
-- Q2 (a)
declare
    v_salary number;
begin
    select salary into v_salary from employees where employee_id = 100;
    if v_salary > 50000 then
        dbms_output.put_line('high salary');
    else
        dbms_output.put_line('normal salary');
    end if;
end;
/s
-- or
```

Script Output x Query Result x

Task completed in 0.06 seconds

normal salary

PL/SQL procedure successfully completed.

Script Output x Query Result x

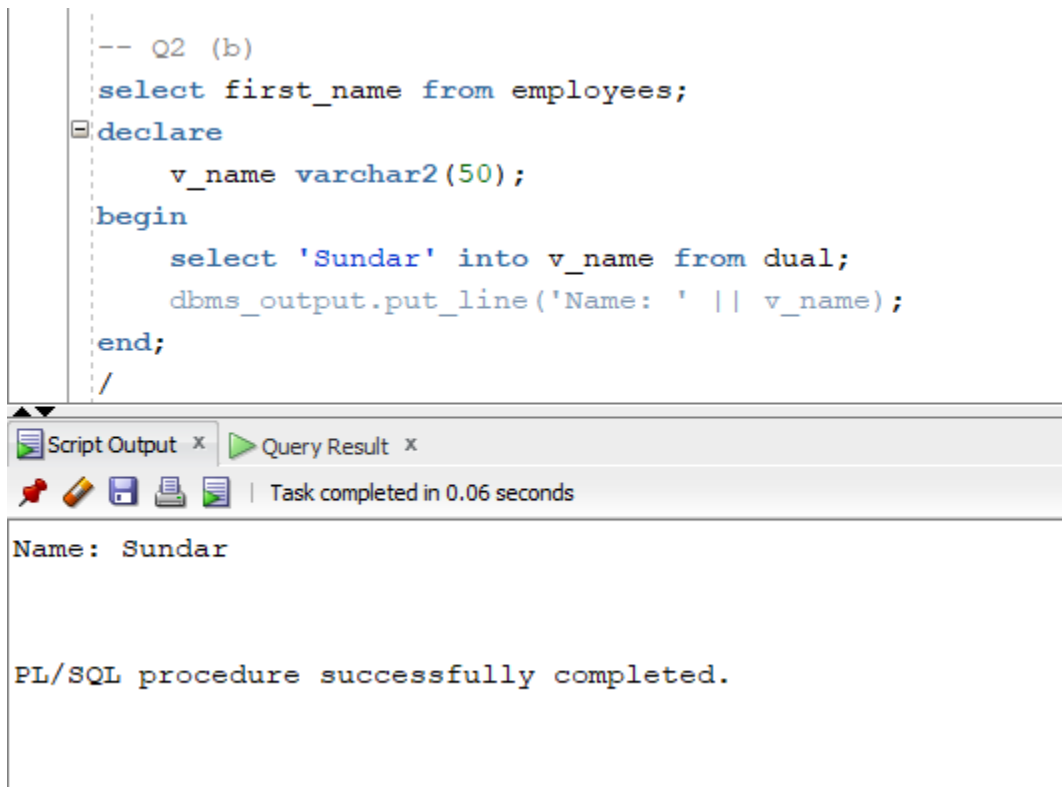
Task completed in 0.054 seconds

PL/SQL procedure successfully completed.

(b) A university system:

- Fetch student name using SELECT INTO
- Display it

```
-- Q2 (b)
select first_name from employees;
declare
    v_name varchar2(50);
begin
    select 'Sundar' into v_name from dual;
    dbms_output.put_line('Name: ' || v_name);
end;
/
```



The screenshot shows a SQL IDE interface. The top pane contains a PL/SQL script for Q2 (b). The script declares a variable v_name of type varchar2(50), selects the value 'Sundar' from the dual table into v_name, and then uses dbms_output.put_line to display the name. The bottom pane shows the script output, which displays 'Name: Sundar' and a confirmation message 'PL/SQL procedure successfully completed.' The task was completed in 0.06 seconds.

Script Output x Query Result x

Task completed in 0.06 seconds

Name: Sundar

PL/SQL procedure successfully completed.

(c) A bank system:

- If balance < 1000 → add 500
 - Else → add 200
- Use IF-ELSE

```
-- Q2 (c)
declare
    v_balance number := &enter_balance;
begin
    if v_balance < 1000 then
        v_balance := v_balance + 500;
    else
        v_balance := v_balance + 200;
    end if;
    dbms_output.put_line('new balance : ' || v_balance);
end;
/

-- Q2 (d)

old:declare
    v_balance number := &enter_balance;
begin
    if v_balance < 1000 then
        v_balance := v_balance + 500;
    else
        v_balance := v_balance + 200;
    end if;
    dbms_output.put_line('new balance : ' || v_balance);
end;

new:declare
    v_balance number := 6000;
begin
    if v_balance < 1000 then
        v_balance := v_balance + 500;
    else
        v_balance := v_balance + 200;
    end if;
    dbms_output.put_line('new balance : ' || v_balance);
end;
new balance : 6200

PL/SQL procedure successfully completed.
```

(d) A company wants to display all employees in department 90
Use a **FOR LOOP** with **SELECT** query

```
-- Q2 (d)
begin
  dbms_output.put_line('Employee in dept 90');
  for emp in (select * from employees where department_id = 90)
  loop
    dbms_output.put_line(emp.first_name || ' ' || emp.last_name);
  end loop;

end;
/

-- Q3 (a)
declare
```

Script Output x Query Result x

Task completed in 0.146 seconds

Employee in dept 90
Steven King
Neena Kochhar
Lex De Haan

PL/SQL procedure successfully completed.

Q3

a) A payroll system:

- If department = 10 → +1000 bonus
 - If department = 20 → +2000 bonus
 - Else → no change
- Use CASE statement

```
-- Q3 (a)
declare
  v_department_id number := 20;
  v_bonus number := 0;
begin
  case v_department_id
    when 10 then
      v_bonus := 1000;
    when 20 then
      v_bonus := 2000;
    else
      v_bonus := 0; -- no change
    end case;
  dbms_output.put_line('department: ' || v_department_id || ' | bonus applied: ' || v_bonus);
end;
/
```

Script Output x Query Result x

Task completed in 0.056 seconds

department: 20 | bonus applied: 2000

PL/SQL procedure successfully completed.

b) A system processes salary:

- If department = 90 AND salary between range → apply commission
Use Nested IF


```
-- Q3 (b)
declare
    v_dept number := 90;
    v_salary number := 8000;
    v_commission number := 0;
begin
    if v_dept = 90 then
        if v_salary between 5000 and 10000 then
            v_commission := v_salary * 0.15; -- applying a 15% commission as an example
            dbms_output.put_line('commission applied successfully: ' || v_commission);
        else
            dbms_output.put_line('no commission.');
```

Task completed in 0.047 seconds

commission applied successfully: 1200

PL/SQL procedure successfully completed.

Q4

Design a stored procedure:

- Input: employee_id
- Output: updated salary
- Logic:
 - Increase salary based on performance tiers
 - Print result

```
-- Q4
create or replace procedure calculate_performance_salary(
    p_employee_id in number,
    p_performance_tier in varchar2,
    p_updated_salary out number
)
is
    v_base_salary number;
begin
    select salary into v_base_salary from employees where employee_id = p_employee_id;
    if p_performance_tier = 'excellent' then
        p_updated_salary := v_base_salary + 2000;
    elsif p_performance_tier = 'good' then
        p_updated_salary := v_base_salary + 1000;
    else
        p_updated_salary := v_base_salary + 500;
    end if;
end;
/

declare
    v_new_salary number;
begin
    calculate_performance_salary(101, 'excellent', v_new_salary);
    dbms_output.put_line('New salary : ' || v_new_salary);
end;
/
```

Script Output x Query Result x

Task completed in 0.046 seconds

Procedure CALCULATE_PERFORMANCE_SALARY compiled

New salary : 20700

PL/SQL procedure successfully completed.

Script Output x Query Result x

SQL | Fetched 50 rows in 0.013 seconds

	FIRST_NAME
1	Ellen
2	Sundar
3	Mozhe
4	David
5	Hermann
6	Shelli
7	Amit
8	Elizabeth
9	Sarah
10	David
11	Laura

Q5.

Design a procedure using IN, OUT, IN OUT parameters:

Scenario:

- **Input: employee salary**
- **OUT: bonus**
- **IN OUT: updated salary**

Logic:

- **Calculate bonus (10%)**
- **Add bonus to salary**
- **Return both**

```
-- Q5
create or replace procedure salary_bonus(
    p_base_salary in number,
    p_bonus out number,
    p_updated_salary in out number
)
is
begin
    p_bonus := p_base_salary * 0.1;
    p_updated_salary := p_base_salary + p_bonus;
end;
/

declare
    v_salary number := 1600;
    v_bonus number;
    v_final_salary number := 0;
begin
    salary_bonus(v_salary,v_bonus,v_final_salary);
    dbms_output.put_line('base salary: ' || v_salary);
    dbms_output.put_line('calculated bonus : ' || v_bonus);
    dbms_output.put_line('updated salary: ' || v_final_salary);
end;
/
```

Script Output x

Task completed in 0.059 seconds

Procedure SALARY_BONUS compiled

base salary: 1600

calculated bonus : 160

updated salary: 1760

PL/SQL procedure successfully completed.